■ PetCare QA

Hands-on Lab: Automação de Testes

Sistema de Agendamento de Banho & Tosa

■ Objetivos de Aprendizagem

- Automatizar casos de teste de API (backend) com Postman, Playwright ou Cypress
- Automatizar casos de teste de interface (frontend) com Playwright ou Cypress
- Implementar validações de formulários, fluxos de autenticação e CRUD
- Praticar boas práticas: data-testid, waits explícitos, isolamento de testes
- Documentar e reportar resultados com evidências

■ Contexto da Aplicação

O PetCare é um sistema web de agendamento de banho e tosa. O projeto tem duas partes: um frontend estático (GitHub Pages) e uma API REST local (Node.js + Express) documentada com Swagger UI.

- Autenticação: login e logout de administrador
- Proteção de rota: redirecionamento automático para login.html
- Agendamentos: CRUD completo via API e interface
- Formulário com validações ricas (campos obrigatórios, data futura, formato)
- Conflito de horário: impede dois agendamentos no mesmo dia/hora
- Filtros: busca por nome do pet e por status
- Estatísticas: totais por status em tempo real

■ Como Rodar o Ambiente

API + Frontend Sistema completo (login + agendamentos	`cd petcare-qa`
	`npm install`
	`npm start`
	
	http://localhost:3001

Swagger UI	http://localhost:3001/docs
JSON para importar no Postman	http://localhost:3001/docs.json
Credenciais	`usuario: admin senha: petcare123`

■ Ferramentas Sugeridas

- Playwright (JavaScript) — frontend e API
- Cypress (JavaScript) — frontend e API
- Postman — exploração manual e automação de API
- Robot Framework + Browser Library

■ Casos de Teste — Backend (API)

Base URL: http://localhost:3001 | Swagger: http://localhost:3001/docs

CT-API-001 Login com credenciais válidas

Objetivo:

Validar autenticação bem-sucedida do administrador.

Endpoint:

```
POST /auth/login
```

Dados de entrada:

```
{
  "usuario": "admin",
  "senha": "petcare123"
}
```

Validações esperadas:

- Status code: 200
- Resposta contém: mensagem: "Login realizado com sucesso"
- Resposta contém objeto usuario com campos usuario e nome
- Campo senha NÃO está presente na resposta
- Tempo de resposta inferior a 2 segundos

CT-API-002 Login com credenciais inválidas

Objetivo:

Validar rejeição de credenciais incorretas.

Endpoint:

```
POST /auth/login
```

Dados de entrada:

```
{
  "usuario": "admin",
  "senha": "senhaerrada"
}
```

Validações esperadas:

- Status code: 401
- Resposta contém: mensagem: "Usuário ou senha incorretos."

CT-API-003 Login com campos ausentes

Objetivo:

Validar que campos obrigatórios são exigidos.

Endpoint:

```
POST /auth/login
```

Dados de entrada:

```
{
  "usuario": "admin"
}
```

Validações esperadas:

- Status code: 400
- Resposta contém: mensagem: "Usuário e senha são obrigatórios."

CT-API-004 Listar todos os agendamentos

Objetivo:

Validar retorno da lista completa de agendamentos.

Endpoint:

```
GET /agendamentos
```

Validações esperadas:

- Status code: 200
- Resposta é um array
- Array contém pelo menos 3 itens (dados de exemplo)
- Cada item contém: id, nomePet, tutor, telefone, servico, porte, data, horario, status, criadoEm
- Campo id é um número inteiro positivo
- Campo status é um de: agendado, concluido, cancelado

CT-API-005 Filtrar agendamentos por status

Objetivo:

Validar que o filtro por status retorna apenas os registros correspondentes.

Endpoint:

```
GET /agendamentos?status=agendado
```

Validações esperadas:

- Status code: 200
- Todos os itens retornados têm status: "agendado"
- Itens com outros status não aparecem na resposta

CT-API-006 Filtrar agendamentos por nome do pet

Objetivo:

Validar busca parcial e case-insensitive por nome.

Endpoint:

```
GET /agendamentos?pet=bol
```

Validações esperadas:

- Status code: 200
- Todos os itens retornados têm nomePet contendo "bol" (ex: "Bolinha")
- Busca é case-insensitive

CT-API-007 Buscar agendamento por ID existente

Objetivo:

Validar retorno de agendamento específico.

Endpoint:

```
GET /agendamentos/1
```

Validações esperadas:

- Status code: 200
- Resposta contém agendamento com id: 1
- Todos os campos obrigatórios estão presentes
- Campo nomePet não está vazio

CT-API-008 Buscar agendamento por ID inexistente

Objetivo:

Validar tratamento de ID não encontrado.

Endpoint:

```
GET /agendamentos/9999
```

Validações esperadas:

- Status code: 404
- Resposta contém: mensagem: "Agendamento não encontrado."

CT-API-009 Criar novo agendamento (sucesso)

Objetivo:

Validar criação de agendamento com todos os dados válidos.

Endpoint:

```
POST /agendamentos
```

Dados de entrada:

```
{
  "nomePet": "Mingau",
  "tutor": "Pedro Costa",
  "telefone": "923456789",
  "servico": "banho",
  "porte": "medio",
  "data": "2026-06-15",
  "horario": "09:00",
  "observacoes": ""
}
```

Validações esperadas:

- Status code: 201
- Resposta contém: mensagem: "Agendamento criado com sucesso"
- Resposta contém objeto agendamento com id gerado automaticamente
- Campo criadoEm foi preenchido automaticamente no formato ISO 8601
- Campo status é "agendado" por padrão
- Valores correspondem aos dados enviados
- GET /agendamentos/:id com o novo id retorna 200

CT-API-010 Criar agendamento com dados inválidos

Objetivo:

Validar rejeição de dados que violam as regras de negócio.

Endpoint:

```
POST /agendamentos
```

Dados de entrada:

```
{
  "nomePet": "A",
  "tutor": "Jo",
  "telefone": "abc",
  "servico": "spa",
  "porte": "gigante",
  "data": "2020-01-01",
  "horario": "13:00"
}
```

Validações esperadas:

- Status code: 400
- Resposta contém: mensagem: "Dados inválidos."
- Resposta contém array erros com um objeto por campo inválido
- Cada erro contém campos campo e mensagem
- Agendamento não é criado

CT-API-011 Criar agendamento com horário ocupado

Objetivo:

Validar que o sistema impede conflito de horário.

Pré-condições:

• Já existe agendamento para 2025-12-20 às 10:00 (dado de exemplo id:1)

Endpoint:

```
POST /agendamentos
```

Dados de entrada:

```
{
  "nomePet": "Thor",
  "tutor": "Marcos Lima",
  "telefone": "912000000",
  "servico": "tosa",
  "porte": "grande",
  "data": "2025-12-20",
  "horario": "10:00"
}
```

Validações esperadas:

- Status code: 400
- Resposta contém: mensagem: "Já existe um agendamento para esta data e horário."
- Resposta contém erro com campo: "horario"

CT-API-012 Atualizar agendamento existente

Objetivo:

Validar atualização parcial de um agendamento.

Endpoint:

```
PUT /agendamentos/1
```

Dados de entrada:

```
{
  "nomePet": "Bolinha Jr.",
  "status": "concluido"
}
```

Validações esperadas:

- Status code: 200
- Resposta contém: mensagem: "Agendamento atualizado com sucesso"
- Campo id permanece 1 (inalterado)
- Campo nomePet foi atualizado para "Bolinha Jr."
- Campo status foi atualizado para "concluido"
- Demais campos permanecem inalterados

CT-API-013 Atualizar agendamento inexistente

Objetivo:

Validar tratamento de atualização com ID inválido.

Endpoint:

```
PUT /agendamentos/9999
```

Dados de entrada:

```
{
  "nomePet": "Teste"
}
```

Validações esperadas:

- Status code: 404
- Resposta contém: mensagem: "Agendamento não encontrado."

CT-API-014 Remover agendamento existente

Objetivo:

Validar exclusão de agendamento pelo ID.

Endpoint:

```
DELETE /agendamentos/2
```

Validações esperadas:

- Status code: 200
- Resposta contém: mensagem: "Agendamento removido com sucesso"
- GET /agendamentos/2 subsequente retorna 404
- GET /agendamentos não inclui mais o item com id: 2

CT-API-015 Remover agendamento inexistente

Objetivo:

Validar tratamento de exclusão com ID inválido.

Endpoint:

```
DELETE /agendamentos/9999
```

Validações esperadas:

- Status code: 404
- Resposta contém: mensagem: "Agendamento não encontrado."

CT-API-016 Obter estatísticas do sistema

Objetivo:

Validar cálculo e retorno dos totais por status.

Endpoint:

```
GET /estatisticas
```

Validações esperadas:

- Status code: 200
- Resposta contém: totalAgendamentos, agendados, concluidos, cancelados, petsUnicos
- Todos os valores são números inteiros não-negativos
- totalAgendamentos = agendados + concluidos + cancelados
- Após criar um agendamento, totalAgendamentos aumenta em 1
- Após deletar um agendamento, totalAgendamentos diminui em 1

■ Casos de Teste — Frontend (Interface)

URL: https://rebvisconti.github.io/petcare-qa/ | Login: admin / petcare123

CT-FE-001 Acesso direto sem sessão — redirecionamento

Objetivo:

Validar que index.html redireciona para login.html quando não há sessão ativa.

Pré-condições:

• sessionStorage limpo

Passos:

1. Limpar o sessionStorage do navegador
2. Acessar diretamente a URL raiz do projeto

Validações esperadas:

- ■ URL muda automaticamente para login.html
- ■ Conteúdo protegido não aparece antes do redirecionamento
- ■ Formulário de login está visível e interativo

CT-FE-002 Login com campos vazios

Objetivo:

Validar que o sistema exige preenchimento dos campos obrigatórios.

Passos:

1. Acessar login.html
2. Deixar ambos os campos em branco
3. Clicar em "Entrar"

Validações esperadas:

- ■ Mensagem de erro: "Informe o usuário."
- ■ Formulário não é submetido
- ■ Nenhum redirecionamento ocorre

CT-FE-003 Login com credenciais incorretas

Objetivo:

Validar mensagem de erro quando credenciais estão erradas.

Dados de entrada:

```
Usuário: admin  |  Senha: senhaerrada
```

Passos:

1. Preencher usuário com "admin"
2. Preencher senha com "senhaerrada"
3. Clicar em "Entrar"

Validações esperadas:

- Mensagem global: "Usuário ou senha incorretos."
- Ambos os campos ficam com borda vermelha
- Campo senha é limpo automaticamente
- Nenhuma sessão criada no sessionStorage

CT-FE-004 Login com credenciais válidas

Objetivo:

Validar fluxo completo de autenticação bem-sucedida.

Dados de entrada:

```
Usuário: admin  |  Senha: petcare123
```

Passos:

1. Preencher usuário com "admin"
2. Preencher senha com "petcare123"
3. Clicar em "Entrar"

Validações esperadas:

- Botão exibe "Entrando..." durante o delay de 500ms
- Botão muda para "✓ Acesso liberado!"
- Redirecionamento para index.html
- sessionStorage contém petcare_sessao com autenticado: true
- Header exibe "■ admin" e botão "Sair"

CT-FE-005 Botão mostrar/ocultar senha

Objetivo:

Validar alternância de visibilidade do campo senha.

Passos:

1. Digitar qualquer texto no campo Senha
2. Clicar no botão ■
3. Clicar novamente no botão

Validações esperadas:

- ■ Após 1º clique: type muda de "password" para "text"
- ■ Ícone muda de ■ para ■
- ■ Após 2º clique: type volta para "password"
- ■ Texto permanece no campo durante toda a alternância

CT-FE-006 Logout — limpeza de sessão

Objetivo:

Validar que o logout encerra a sessão e redireciona para login.

Pré-condições:

• Usuário autenticado

Passos:

1. Clicar no botão "Sair" no header

Validações esperadas:

- ■ Redirecionamento imediato para login.html
- ■ sessionStorage não contém mais petcare_sessao
- ■ Tentativa de acessar index.html redireciona para login

CT-FE-007 Submissão do formulário vazio

Objetivo:

Validar que todos os campos obrigatórios exibem erro ao submeter em branco.

Pré-condições:

• Usuário autenticado

Passos:

1. Sem preencher nenhum campo, clicar em "Agendar"

Validações esperadas:

- Erros em: Nome do pet, Tutor, Telefone, Serviço, Porte, Data, Horário
- Todos os campos com erro ficam com borda vermelha
- Nenhum agendamento é criado

CT-FE-008 Validação — data no passado

Objetivo:

Validar que datas anteriores a hoje são rejeitadas.

Pré-condições:

• Usuário autenticado

Passos:

1. Preencher todos os campos corretamente
2. Injetar data passada via JS: document.getElementById("data").value = "2020-01-01"
3. Clicar em "Agendar"

Validações esperadas:

- Erro exibido: "A data não pode ser no passado."
- Agendamento não é criado

Dica: o campo date tem min= definido para hoje — injete o valor via DevTools ou diretamente no teste.

CT-FE-009 Criar agendamento com sucesso (happy path)

Objetivo:

Validar criação completa com todos os dados válidos.

Pré-condições:

• Usuário autenticado

• Horário escolhido disponível

Passos:

1. Preencher Nome do pet: "Bolinha"
2. Preencher Tutor: "Ana Lima"
3. Preencher Telefone: "912345678"
4. Selecionar Serviço: "Banho + Tosa"
5. Selecionar Porte: "Pequeno"
6. Selecionar data futura válida
7. Selecionar Horário: "10:00"
8. Clicar em "Agendar"

Validações esperadas:

- Mensagem: "Agendamento criado com sucesso!"
- Formulário é limpo após criação
- Card aparece na lista com todos os dados corretos
- Badge exibe "Agendado"
- Contador atualiza corretamente
- Após reload, agendamento ainda está presente

CT-FE-010 Conflito de horário

Objetivo:

Validar que o sistema impede dois agendamentos no mesmo dia/hora.

Pré-condições:

• Agendamento existente para a data/horário escolhidos

Passos:

1. Criar um agendamento para data e horário específicos
2. Tentar criar segundo agendamento para a mesma data e horário

Validações esperadas:

- Mensagem: "Já existe um agendamento para esta data e horário."
- Campo Horário fica com borda vermelha
- Segundo agendamento não é criado

CT-FE-011 Editar agendamento existente

Objetivo:

Validar fluxo completo de edição.

Pré-condições:

• Pelo menos 1 agendamento criado

Passos:

1. Clicar em "Editar" no card
2. Verificar que formulário foi preenchido com os dados do card
3. Alterar Nome do pet para "Bolinha Jr."
4. Clicar em "Salvar alterações"

Validações esperadas:

- Formulário carrega com dados existentes
- Botão muda para "Salvar alterações"
- Mensagem: "Agendamento atualizado com sucesso!"
- Card reflete o novo nome
- ID permanece o mesmo

CT-FE-012 Excluir agendamento com confirmação

Objetivo:

Validar remoção com diálogo de confirmação.

Pré-condições:

• Pelo menos 1 agendamento criado

Passos:

1. Clicar em "Excluir" no card
2. Confirmar a exclusão no diálogo

Validações esperadas:

- Diálogo de confirmação exibido antes da exclusão
- Mensagem: "Agendamento excluído."
- Card removido da lista imediatamente
- Contador decrementado
- Após reload, agendamento não aparece mais

CT-FE-013 Busca por nome do pet em tempo real

Objetivo:

Validar que o filtro filtra a lista dinamicamente.

Pré-condições:

• Agendamentos para "Bolinha" e "Rex" criados

Passos:

1. Digitar "Bol" no campo de busca

Validações esperadas:

- Apenas o card de "Bolinha" é exibido
- Card de "Rex" some da lista
- Ao limpar o campo, ambos reaparecem
- Busca é case-insensitive

CT-FE-014 Alterar status do agendamento na edição

Objetivo:

Validar que o campo status aparece apenas no modo edição e que a alteração é refletida no card.

Pré-condições:

• Usuário autenticado

• Pelo menos 1 agendamento com status "Agendado"

Passos:

1. Criar um novo agendamento — verificar que o campo Status NÃO aparece
2. Clicar em "Editar" no card recém criado
3. Verificar que o campo Status aparece pré-preenchido com "Agendado"
4. Alterar Status para "Concluído"
5. Clicar em "Salvar alterações"

Validações esperadas:

- Campo Status NÃO está visível durante a criação de agendamento
- Campo Status aparece ao entrar em modo edição
- Campo Status é pré-preenchido com o status atual do agendamento
- Após salvar: badge do card exibe "Concluído" com cor verde
- Após salvar: a barra lateral esquerda do card muda para verde
- Ao cancelar a edição: campo Status some do formulário

CT-FE-015 Filtro por status na lista

Objetivo:

Validar que o select de filtro exibe apenas agendamentos do status selecionado.

Pré-condições:

• Agendamentos com status diferentes existentes na lista

Passos:

1. Selecionar "Agendado" no select de filtro
2. Selecionar "Concluído" no select de filtro
3. Selecionar "Todos os status" no select de filtro

Validações esperadas:

- Filtro "Agendado": apenas cards com badge "Agendado" são exibidos
- Filtro "Concluído": apenas cards com badge "Concluído" são exibidos
- Filtro "Todos os status": todos os cards reaparecem
- Contador atualiza corretamente conforme o filtro aplicado

■ Entregáveis

- Código dos testes automatizados (API e frontend)
- Relatório de execução com evidências (screenshots, logs)
- README com instruções de execução dos testes

■ Dicas para o Sucesso

Comece pelo happy path: CT-API-001, CT-API-009 e CT-FE-004 primeiro — login e criação funcionando antes dos cenários negativos.

Reinicie a API para resetar dados: Os dados ficam em memória. Reiniciar o servidor (Ctrl+C → npm start) restaura os 3 agendamentos de exemplo — ideal para isolar testes.

Use data-testid no frontend: Todos os elementos têm atributos data-testid. Use-os como seletores — estáveis mesmo com mudanças de estilo.

Waits explícitos na UI: O login tem delay de 500ms intencional. Use waitForSelector ou expect(...).toBeVisible() em vez de waitForTimeout.

Importe o Swagger no Postman: Acesse http://localhost:3001/docs.json e importe via Import → Link no Postman para ter toda a coleção gerada automaticamente.

Documente bugs encontrados: Se encontrar comportamento inesperado, registre: passos para reproduzir, resultado esperado e resultado atual.

■ Seletores data-testid — Referência Rápida

Seletor	Elemento
`[data-testid="input-usuario"]`	Campo usuário (login)
`[data-testid="input-senha"]`	Campo senha (login)
`[data-testid="btn-entrar"]`	Botão Entrar
`[data-testid="mensagem-login"]`	Mensagem de erro do login
`[data-testid="btn-logout"]`	Botão Sair (header)
`[data-testid="input-nome-pet"]`	Campo nome do pet
`[data-testid="input-tutor"]`	Campo tutor
`[data-testid="input-telefone"]`	Campo telefone
`[data-testid="select-servico"]`	Select serviço
`[data-testid="select-porte"]`	Select porte
`[data-testid="input-data"]`	Campo data
`[data-testid="select-horario"]`	Select horário

`[data-testid="btn-agendar"]`	Botão Agendar / Salvar
`[data-testid="btn-cancelar"]`	Botão Cancelar
`[data-testid="mensagem-global"]`	Mensagem sucesso/erro (form)
`[data-testid="lista-agendamentos"]`	Container da lista
`[data-testid="card-agendamento"]`	Card individual (múltiplos)
`[data-testid="btn-editar"]`	Botão Editar (no card)
`[data-testid="btn-excluir"]`	Botão Excluir (no card)
`[data-testid="contador-agendamentos"]`	Contador de agendamentos
`[data-testid="input-busca"]`	Campo de busca
`[data-testid="lista-vazia"]`	Mensagem lista vazia
`[data-testid="erro-nome-pet"]`	Mensagem erro nome pet
`[data-testid="erro-data"]`	Mensagem erro data
`[data-testid="erro-horario"]`	Mensagem erro horário